This problem set is not going to be collected, although solutions will be posted before the quiz. I recommend you treat this as if this were due Tuesday of week 5 at lecture. In addition to solving them, I recommend writing up solutions as if you were going to submit them. Unlike a problem set collected for credit, though, you are free to discuss with classmates and course staff and get help.

I recommend you first attempt the problems on your own, then discuss with classmates. After that, you may want to consult course staff at help hours or on Ed Discussion. After solutions are posted, consult the solutions and compare to what you wrote, and then discuss the differences with your study group.

This set is intended to help students who want to bolster their recursive problem solving skills before moving onto dynamic programming.

1. The McDonald's near campus sells Chicken McNuggets in orders of 6, 9, or 20. Suppose you are ordering for a party and you know exactly how many McNuggets will be eaten by guests. It turns out that, for any integer $n \geq 44$, you can order exactly n Chicken McNuggets at this McDonald's.

   For purposes of this problem, you cannot throw out McNuggets or allow them to go uneaten, such as by acquiring $n = 44$ by buying two twenty packs and a six pack, then discarding two. If the thought of this many Chicken McNuggets is too disgusting, you may pretend you are buying $n \geq 44$ celery sticks in bunches of 6, 9, or 20 (feel free to rename the function below in that case).

   Finish the recursive function below to complete the ordering and return the counts by reference parameters. You may assume for this problem that there will be no overflow or underflow at any point in the problem and that stack space is not a concern. The code has been started for you and is part of a correct solution.

```
void buyChicken (unsigned n, unsigned & num6Packs, unsigned & num9Packs,
                 unsigned & num20Packs)
{
  if( 44 == n ){
    num20Packs = 1;  num6Packs = 4;  num9Packs = 0;
  }else if (45 == n ){
    num20Packs = 0; num6Packs = 3;  num9Packs = 3;
  }else if (46 == n ){
    num20Packs = 2; num6Packs = 1; num9Packs = 0;
  }
```

   More questions are on the next digital page

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

2. *Skittles* are small candies; each piece of candy is one of five flavors (grape (**P**urple), **L**emon, **O**range, **S**trawberry, or **G**reen). An exciting new party game, *Skittle Trader*, is sweeping the nation, and the skills you are learning in I&C SCI 46 will help you to achieve a high score!

   Here are the rules of the game. You are at a party with n of your friends. Each of you has one piece of *Skittles* candy. Your friends form a line. You walk down the line, visiting friends in order. At each friend, you have a choice: do you want to trade the candy you are holding for the one they are holding?

   - If you do not trade candy, your score remains the same, unless you had the same flavor candy, in which case, you earn one point.
   - If you and the friend trade, but you had different flavor types, you lose one point. Your score *can* go negative.

   You begin the game with zero points. Your goal is to finish the game with the most possible points.

   For purposes of the input to this problem, the Skittles flavors are **char**s from the set {'P', 'L', 'O', 'S', 'G'}. The input is represented as a Linked List with the following node declaration:

```
struct Node
{
  Node(char c, Node * n) : letter(c), next(n) {}
  char letter;
  Node * next;
};
```

   Note that we are returning an **int** because points can be negative. The second parameter is the type of Skittle you are holding at the beginning. Your running time does not need to be good and you do not need to tell us the running time. Simple and correct code is strongly preferred.

   You do not need to write this in C++; pseudo-code is fine. C++ is provided for clarity, not as a requirement.

```
int skittleTrader(Node * front, char mySkittle)
```

More questions are on the next digital page

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

3. Imagine you have almost finished the very difficult ten-weeks of Fall Quarter 2024. All that remains is finals week. You have H hours available to study for your finals; for this problem, assume that you always spend a non-negative integer-numbered hours studying for any given course (including zero, which is an integer). You have figured out, for each course, a "study benefit" : you know, for each $i \in [0, H]$, if you study for i hours, you will get grade study[i] (which here is a C++ **double**, representing the grade points: if i hours studying would get you an A, then study[i] = 4.0, an A- then study[i] = 3.7, etc).

   For this problem, you want to write a function that takes, as input, the front node in a linked list of your courses. Each course has a pointer to the next one in your schedule (which will be `nullptr` if this is the last course in the list) and a `std::vector` of size at least H that tells you the points you'd get for studying any particular number of hours. Your goal is to figure out the maximum total grade points you can get this quarter, given that you can spend up to H hours studying.

   Hint: You should study somewhere between 0 and H hours for each course.

   It is allowed (but not required) to assume that for all i, if you study for $i+1$ hours for a given course, you get more points than if you study for i hours for that course. It is also allowed to assume that if you study zero hours for a course, you get zero points.

```
struct course {
    course * next;
    std::vector<double> study;
};
double bestStudy(course * first, unsigned H){
```

4. (2 points) Consider the following definition for a node in a linked list:

```
struct Node
{
  Node(unsigned v, Node * n) : value(v), next(n) {}
  unsigned value;
  Node * next;
};
```

   Suppose we have a linked list of these and wish to answer a question: is there a subset of these Nodes such that the combined value equals a particular value 'target'?

   Complete the following function that returns a bool value to indicate the answer to the question. You may assume that the initial call gives a link to the front of the linked list and that no overflow or underflow will occur at any point in the run of a program.

   You *may* write in your choice of C++ or pseudo-code. You *do not* need to delete the list when you are done. If you elect to write in C++, we will not deduct points for minor syntax errors, as long as your intent is clear.

   Hint: if there is a subset of the Nodes whose combined value adds to the target, then either the first element is in that subset or it isn't.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

```
bool subsetSum(Node * front, unsigned target)
{
```

5. On the previous question, we saw the *Subset Sum* problem, in which we had a linked list of unsigned integers and asked if a subset added up to some parameter T. If you have not already done so, please review that question and solution before continuing.

   Now, imagine you want to answer a different question with the same input. Instead of asking if there is a subset that adds up to T, you want to know *how many distinct subsets* add up to T. For example, if your input is $\{2, 3, 4, 5\}$ with $T = 9$, there are two distinct subsets ($\{2, 3, 4\}$ and $\{4, 5\}$) that add up to 9. Note that 4 could be reused between subsets, but not within one (i.e., $\{3, 3, 3, \}$ is not a subset that adds up to 9).

6. Suppose we are given an n-digit positive integer X. We repeatedly remove one digit from either end of X (your choice). The *square depth* of X is the maximum number of perfect squares that you can see during this process.

   For example, the number 32492 has square-depth 3, by the following sequence of removals: remove the 2 at the end, leaving us with 3249 (which is 57^2). Then remove the 9 from the end, leaving us with 324 (which is 18^2). Then remove the 3 from the front, leaving us with 24, which is not a perfect square. Then remove the 2 from the front, leaving us with 4 (which is 2^2). The last removal leaves us with an empty number, and we are done.

   Complete the following function to compute the square depth of a number, represented as a vector of digits. The function returns the square depth of the number represented by `digits[i]` through `digits[j]` inclusive, and is called with squareDepth(0, digits.size()) initially.

   You may assume you have access to a helper function UNSIGNED ISSQUARE(DIGITS, i, j) that returns 1 if digits $[i \ldots j]$ is a perfect square, and zero otherwise. This function runs in time $\mathcal{O}(1)$.

```
unsigned squareDepth(std::vector<unsigned> digits, size_t i, size_t j)
```

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.